

Sinjan Mishra

Roll 2405910 · GitHub `Thesin-01` · CSE-14 · Reviewer: Shaili Seth (Sprint A), Sohan Mandal (Sprint B)

You own time. Every protocol must decide how long to wait before assuming a packet is lost, and what to resend when it gives up. You build the timer machinery all three protocols share, the Jacobson/Karels estimator that sets the timeout from measured round-trip times, and Go-Back-N itself. In week 6 you run Experiment 4 — the one experiment no other group will have. **Nothing in Sprint A blocks you:** your timer heap and estimator are plain Java, so you can build and test them before the packet format even exists.

Fri 18 Sep

`TimerWheel` merged — Sohini needs it for Selective Repeat

Wed 23 Sep

Go-Back-N merged, passing 5 profiles × 10 seeds

Thu 24 Sep

Adaptive RTO working in all three protocols

YOU DEPEND ON	DEPEND ON YOU
Shaili — <code>Packet</code> on Thu 17; <code>ArqProtocol</code> and conventions on Sat 19	Sohini — <code>TimerWheel</code> on Fri 18
Sohan — emulator on Thu 17; trace lines with <code>type</code> and <code>ack</code> for Experiment 4	Shaili, Sohini — adaptive RTO on Thu 24
	Sohan — the <code>--rto</code> flag for the experiments on Fri 25

Contents

1 · Get set up	9 · Experiment 4
2 · Your part of the system	10 · Your report section
3 · Concepts you must master	11 · How we work together
4 · Design decisions you must defend	12 · Using AI without losing marks
5 · Your day-by-day plan	13 · Viva preparation
6 · Team conventions	14 · Help, references, glossary
7 · How to test your work	15 · Sprint checklist
8 · When something breaks	

1 · Get set up — by Tuesday 15 September, 10 pm

Gate 1 is the first check of the sprint: all four of us have cloned the repository, verified our git identity, and pushed a first commit. Everything after it depends on this.

The repository is not a shared folder. Sohan's copy lives on Sohan's laptop. You make your own complete copy on *your* laptop by cloning it from GitHub, then you work, commit and push from there. GitHub is only the meeting point. Every command in this handbook runs on your own machine. (If you already have a working clone, start at step 4.)

Step 1 — Install the tools

- **Git.** On Windows this also installs **Git Bash** — run every command in this handbook there, not in PowerShell or Command Prompt.
- **JDK 21** (Adoptium Temurin). Check that `java -version` and `javac -version` both report 21.
- **WSL2 with Ubuntu** on Windows, because the experiments run on Linux. Inside Ubuntu install the JDK and Python too:

```
ws1 --install -d Ubuntu
ws1 -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version
```

Step 2 — Clone onto your laptop

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

You now have your own `rdt-udp` folder. Everything from here on happens inside it.

Step 3 — Tell git who you are, inside that folder

```
git config user.name "Sinjan Mishra"
git config user.email "the-email-on-your-github-account"
```

This email decides whether you count as a contributor. It must be one of the addresses listed at github.com/settings/emails for your account `Thesin-01`. If you would rather not use a personal address, that page shows a private one ending in `@users.noreply.github.com` that links just as well. Commits with an unrecognised email show a grey avatar and are credited to nobody — and sir treats commit history as evidence of individual contribution.

Step 4 — Confirm the build

```
./build.sh
```

You should see three test suites and **33 tests, 0 failed**. If anything fails, paste the output in the group chat the same evening.

Step 5 — Push your first commit

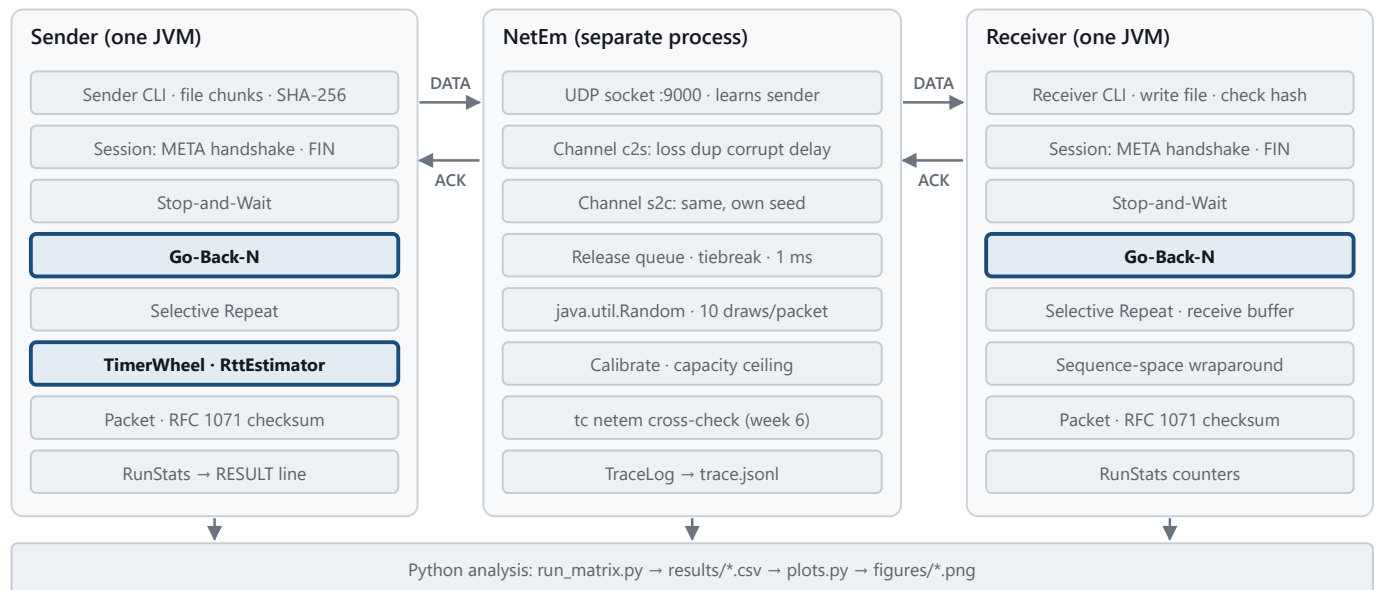
Open `AI-USE.md` and replace `__[name]_` beside your role heading with your name. Then:

```
git checkout -b setup/sinjan
git add AI-USE.md
git commit -m "Add Sinjan to AI-USE.md"
git push -u origin setup/sinjan
git log -1 --format='%an <%ae>'
```

The last command must print your own name and your GitHub email. Open the pull request on GitHub and ask Shaili Seth to approve it. Your first push asks you to sign in to GitHub; on Windows a browser window opens, which is normal.

Done when: your PR is merged, `git log` shows your own name and email, and you can see yourself on the repository's Contributors page.

2 · Your part of the system



Bold, coloured boxes are yours. Grey boxes belong to teammates — either you depend on them, or they depend on you. Packets always travel through NetEm: the sender and receiver never talk directly.

Files you own

FILE	STATUS	WHAT IT DOES
docs/timers-design.md	New, Wed 16	Half-page design note: the event loop, the heap, cancellation, why no threads
src/main/java/rdt/TimerWheel.java	New, Wed 16 – Fri 18	A single-threaded timer heap shared by Go-Back-N and Selective Repeat
src/main/java/rdt/RttEstimator.java	New, Sat 19 – Sun 20	Jacobson/Karels RTT estimation, backoff, RTO floor and cap
src/main/java/rdt/GoBackN.java	New, Mon 21 – Wed 23	Sliding window, cumulative ACKs, one timer on the window base
src/test/java/rdt/TimerWheelTest.java, RttEstimatorTest.java, GoBackNTest.java	New, with each class	See section 7
docs/exp4-design.md	New, Sun 27	How spurious retransmissions are labelled from the trace
analysis/spurious.py	Week 6, with Sohan	The labelling analysis for Experiment 4

On Thu 24 you wire adaptive RTO into `StopAndWait` (Shaili's) and `SelectiveRepeat` (Sohini's). Do that sitting with each owner, not by editing their files alone.

3 · Concepts you must master

3.1 · Why a timeout is a trade-off

Set the timeout too short and the sender resends packets that were never lost — *spurious retransmissions*, which waste the link. Set it too long and, after a real loss, the sender sits idle. Round-trip times vary from packet to packet, so any fixed value is wrong somewhere. That is the whole reason RTO is *estimated*.

3.2 · Which clock

Use `System.nanoTime()` for every timer and every RTT measurement. It is monotonic: it only moves forward.

`System.currentTimeMillis()` is wall-clock time, which jumps when the system clock is corrected — a jump backwards gives a negative RTT. `nanoTime` values are only meaningful as differences, never as a time of day.

3.3 · One thread, one loop

Every protocol runs a loop like this, and your timer heap is what drives it:

1. Ask the heap for the earliest deadline.
2. Set the socket receive timeout to the time remaining — **at least 1 ms**, because `setSoTimeout(0)` means *wait forever*.
3. Block in `receive`. Either a packet arrives, and you handle it (which may cancel or schedule timers), or `SocketTimeoutException` is thrown and you carry on.
4. Fire every timer whose deadline has passed.
5. Go back to step 1.

Why there is no race. Handling an ACK and firing a timeout happen on the same thread, one after the other, so there is never a moment when both run. With a separate timer thread, an ACK could arrive while that thread is halfway through a retransmission, and every piece of window state would need a lock — the exact concurrency the brief warns about.

Granularity. `setSoTimeout` works in milliseconds, so deadlines finer than 1 ms round up. Our round trips are tens of milliseconds, so this is fine. Calling `setSoTimeout` is a system call, so only call it when the value changes. The emulator's loop uses exactly this design — read `NetEm.run` and `blockMillis` to see it working.

Why not a Selector. A selector multiplexes many sockets; a protocol has one, so it would add nothing. It also failed outright on Sohan's Windows laptop, where the JDK could not create its internal loopback pipe.

3.4 · The timer heap

- A `PriorityQueue` ordered by deadline, with an insertion counter as a tiebreak so timers due at the same instant fire in the order they were scheduled. Without the tiebreak the order is undefined, and runs stop being reproducible.
- **Cancelling.** `PriorityQueue.remove(timer)` searches the whole queue — $O(n)$. The alternative is **lazy cancellation**: mark the timer cancelled and simply skip it when it reaches the top — $O(1)$ to cancel. With at most a window's worth of timers the leftover cancelled entries cost nothing. Pick one and be able to justify it.
- **Never change a deadline while the timer is in the queue.** The heap does not notice, its ordering silently breaks, and timers fire late. To reschedule, cancel and add a new entry.
- **When firing:** collect the expired timers first, then run their actions. An action that schedules a new timer while you are still iterating the queue will otherwise throw `ConcurrentModificationException`.

A naming trap you will be asked about. The class is called `TimerWheel`, but it is a heap. A real *hashed timing wheel* is a different structure — buckets by time slot, $O(1)$ insert — used in operating system kernels and in Netty. We chose a heap because we have few timers and want exact ordering. Either say that confidently, or propose renaming it `TimerHeap` at the Friday call.

3.5 · Jacobson/Karels — RFC 6298

On the **first** RTT measurement R :

$$\begin{aligned} \text{SRTT} &= R \\ \text{RTTVAR} &= R / 2 \\ \text{RTO} &= \text{SRTT} + \max(G, 4 \times \text{RTTVAR}) \end{aligned} \quad G = \text{clock granularity, 1 ms here}$$

On every **later** measurement — **in this order**, because the RTTVAR line uses the *old* SRTT:

```
RTTVAR = 3/4 × RTTVAR + 1/4 × |SRTT - R|    (β = 1/4)
SRTT   = 7/8 × SRTT   + 1/8 × R              (α = 1/8)
RTO    = SRTT + max(G, 4 × RTTVAR)
```

Worked example — these exact numbers belong in your tests:

SAMPLE R (MS)	SRTT	RTTVAR	RTO
100 (first)	100	50	300
120	102.5	42.5	272.5
80	99.6875	37.5	249.6875
200	112.2266	53.2031	325.0391

The order trap. If you update SRTT first, the second row becomes RTTVAR 41.875 and RTO 270 instead of 272.5. A test using the table above catches it immediately. Note that the playbook lists the SRTT formula first for readability — follow the RFC's order in code.

Why 4 × RTTVAR. A timeout set at the average RTT would fire on roughly half of perfectly normal packets. Adding a multiple of the variation pushes the timeout into the tail of the distribution, so it fires on genuine loss.

Backoff. Each timeout doubles RTO, up to a cap. This stops a sender hammering a path that is dropping everything.

Floor and cap — a deliberate deviation. RFC 6298 recommends a minimum RTO of **1 second** for the Internet. On our emulated 40 ms path that would stall every loss for 25 round trips and make adaptive RTO look terrible. Choose a lower floor and a cap, write both in `docs/timers-design.md`, and state the deviation and its reason in the report. An examiner will respect a documented deviation; an unexplained one looks like a mistake.

3.6 · Karn's algorithm

```
t = 0 ms      send packet P          RTO = 300 ms
t = 300 ms    timer fires, resend P   RTO backs off to 600 ms
t = 320 ms    an ACK for P arrives

Was it for the original?    RTT = 320 ms
Or for the retransmission?  RTT =  20 ms    - sixteen times smaller
```

The sender cannot tell. Feeding either number into the estimator corrupts it. **Karn's rule: never take an RTT sample from a packet that was sent more than once.** Keep the backed-off RTO, and only resume sampling from a packet that was sent exactly once and then acknowledged. In practice: record each packet's first send time and whether it was ever retransmitted, and sample only when the acknowledged packet was not.

3.7 · Go-Back-N

The sender keeps `base` (oldest unacknowledged) and `nextSeq` (next to send), with window size N.

EVENT	SENDER ACTION
Room in the window and data left	Send <code>nextSeq</code> and advance it. If nothing was outstanding, start the timer.
ACK <i>n</i> , inside the window	Set <code>base = n + 1</code> . If <code>base == nextSeq</code> , stop the timer; otherwise restart it for the new base. Take an RTT sample if Karn allows.
ACK <i>n</i> , below <code>base</code>	A duplicate — ignore it, count it in <code>dup_acks</code>
Timeout	Resend every packet from <code>base</code> to <code>nextSeq - 1</code> , restart the timer, back off RTO

RECEIVER EVENT	RECEIVER ACTION
The expected packet, intact	Deliver it; ACK it; expect the next one
Any other intact packet	Discard it; re-ACK the last packet delivered in order
A corrupt packet	Drop it silently

The receiver keeps **no buffer**. That is what makes Go-Back-N simple, and it is exactly why it loses to Selective Repeat under loss: one lost packet forces everything behind it to be sent again.

3.8 · Why Go-Back-N needs $W \leq 2^k - 1$

With k sequence bits there are 2^k numbers, and they wrap. The sender's window and the receiver's window (size 1) must never be able to overlap: $W + 1 \leq 2^k$.

$k = 2$, numbers 0 1 2 3. Illegal: $W = 4$
 send 0 1 2 3 → all arrive and are delivered → receiver now expects 0 (a NEW packet)
 every ACK is lost → sender times out, resends the OLD 0
 receiver accepts old 0 as new 0 → wrong data, silently

Legal: $W = 3$
 send 0 1 2 → delivered → receiver expects 3
 every ACK is lost → sender resends old 0 1 2 → none is 3 → discarded, re-ACK 2 ✓

3.9 · Fixed RTO mode

`--rto fixed:X` sets RTO to $X \times \text{base RTT}$ (convention 7) with **no estimation and no backoff** — genuinely fixed, so Experiment 4 compares one clean policy against another.

4 · Design decisions you must defend

DECISION	WHY
One thread and a heap, no <code>ScheduledExecutorService</code>	No race between timeouts and ACKs, no locks, reproducible ordering. See 3.3.
Lazy cancellation (or $O(n)$ removal)	Whichever you choose, the cost argument in 3.4.
<code>nanoTime</code> , not wall-clock time	Monotonic; cannot produce negative intervals. See 3.2.
RFC 6298 order, with a lower floor	Correct estimation; the 1-second floor is tuned for the Internet, not a 40 ms emulated path. Documented.
No samples from retransmitted packets	Karn's ambiguity, 3.6.
One Go-Back-N timer, on the base	Everything behind the base is resent together anyway, so one timer is enough.
Cumulative ACK = last in-order packet received	The textbook convention, and team convention 3.
Fixed RTO means no backoff either	Experiment 4 must compare policies, not a mixture of them.

5 · Your day-by-day plan

Sprint A · 14–20 September

Mon 14 – Tue 15

Setup and reading

1. Complete section 1. First commit merged by Tuesday 10 pm.
2. Read **RFC 6298** in full — it is short. Sections 2 and 5 are the ones you implement.
3. Read Kurose & Ross section 3.4.3 (Go-Back-N) and section 3.5.3 (estimating RTT).
4. Work the table in 3.5 by hand before looking at it. Then work it again in the wrong order and see it change.

Done when: your first commit is merged and your hand-worked table matches 3.5.

Wed 16 – Fri 18

TimerWheel · Sohini needs it Friday

1. Wednesday morning: write `docs/timers-design.md` — the loop from 3.3, the heap and your cancellation choice from 3.4, and why there is no timer thread. Commit it.
2. `git checkout -b m2/timer-wheel`. The heap needs four things: schedule a timer for a deadline, cancel a timer, report the earliest live deadline, and hand back every timer that has expired by a given time.
3. Write the tests in section 7 alongside the code, not after.
4. Thursday: review Shaili's `Packet` PR with three *why* questions.
5. Friday: PR by the afternoon, merged by the evening. **8 pm: conventions call** — you own convention 7 and are consulted on 3; raise the `TimerWheel` naming question.

Done when: merged by Friday night, passing a randomised test against a sorted-list oracle.

Be able to explain: questions 1 and 2 in section 13.

Sat 19 – Sun 20

RttEstimator

1. `git checkout -b m2/rtt-estimator`. It needs to accept a sample, react to a timeout with backoff, and report the current RTO, applying your floor and cap.
2. Encode the whole table from 3.5 as a test, plus a test that would fail if the update order were wrong.
3. Decide the floor and the cap, and record them in your design note.

Done when: merged by Sunday night, matching the table to 1e-9. Gate 3 includes it.

Be able to explain: questions 3, 4, 8 and 9 in section 13.

Sprint B · 21–27 September

Mon 21 – Wed 23

Go-Back-N

1. **Monday:** `git checkout -b m2/go-back-n`. Build the sender from the first table in 3.7 against Shaili's `ArqProtocol1`, driven by your heap and estimator, with Karn's rule applied.
2. **Tuesday:** the receiver from the second table. First end-to-end transfers at 0% and 10% loss through the emulator.
3. **Wednesday:** all five integrity profiles × 10 seeds; a wraparound run with `--seqbits 3` and window 7; and the illegal-window demonstration from 3.8. Merged by Wednesday night.

Done when: merged by Wednesday 10 pm, passing 5 profiles × 10 seeds — your part of Gate 4.

Be able to explain: questions 5, 6 and 7 in section 13.

Thu 24 – Fri 25

Adaptive and fixed RTO everywhere

1. Thursday: with Shaili, then Sohini, wire `RttEstimator` into Stop-and-Wait and Selective Repeat.
2. Friday: `--rto fixed:X` for all three. Test that `fixed:1.5` with `--both delay=20` produces an RTO of 60 ms.

Done when: every protocol accepts both `--rto adaptive` and `--rto fixed:1.5`.

Sat 26 – Sun 27

Support runs; design Experiment 4

1. Help keep the overnight experiment runs healthy.
2. Write `docs/exp4-design.md` from section 9 — especially the labelling method.

After the sprint

28 Sep – 26 Oct

Buffer, then weeks 6–8

1. **28 Sep – 5 Oct:** finish anything that slipped; draft your report section.
2. **6–12 Oct:** run Experiment 4 and write `analysis/spurious.py` with Sohan. Teach your module at cross-teaching.
3. **13–19 Oct:** finish your report section.
4. **20–26 Oct:** code freeze 22 October; viva drills.

6 · Team conventions — agreed on the Friday 18 call

Four people are writing code that has to plug together on Monday 21. Each item below is a decision that, left unspoken, becomes a day of integration bugs. **On Friday 18 September at 8 pm, all four of us join a 30-minute call**, confirm or change each default, and Shaili commits the result as `docs/CONVENTIONS.md` on Saturday morning. Until then, treat these as proposals — and if you disagree with one, bring the reason to the call rather than quietly doing something else.

#	TOPIC	PROPOSED DEFAULT	OWNER
1	Protocol interface	One <code>ArqProtocol</code> interface with two operations: <i>send a file to a peer</i> and <i>receive a file on a port</i> . Both take a shared <code>ProtocolConfig</code> (window, RTO mode, sequence-space bits, payload size) and return <code>RunStats</code> . The session layer — metadata handshake, FIN teardown, hash check — sits outside the protocols and wraps whichever one is chosen, so Go-Back-N and Selective Repeat never reimplement it.	Shaili
2	Sequence numbers	The handshake is a DATA packet with <code>seq = 0</code> and flag bit <code>0x01</code> (META) set. File data starts at <code>seq = 1</code> . The sequence space is k bits, set with <code>--seqbits</code> ; the default is 32 and only wraparound tests make it small. Every comparison between sequence numbers uses modular arithmetic.	Shaili
3	What an ACK number means	The <code>ack</code> field always holds a sequence number that <i>was received</i> . Stop-and-Wait and Selective Repeat acknowledge exactly that packet. Go-Back-N's <code>ack n</code> is cumulative: everything up to and including n arrived in order — the textbook convention. Each protocol restates its rule in its class comment.	Shaili, Sinjan, Sohini
4	Payload size	1400 bytes per DATA packet; the last one is shorter. An 8 MiB file is therefore 5,992 data packets plus the handshake.	Shaili
5	Command-line flags	Sender: <code>--file</code> <code>--to host:port</code> <code>--protocol stopwait gbn sr</code> <code>--window N</code> <code>--rto adaptive fixed:X</code> <code>--seqbits k</code> <code>--verbose</code> . Receiver: <code>--port</code> <code>--out</code> <code>--protocol</code> <code>--window N</code> <code>--seqbits k</code> <code>--verbose</code> . An unknown flag is an error, never silently ignored.	Shaili
6	Ports	For manual runs the emulator listens on 9000 and the receiver on 9001. The experiment harness picks fresh ports for every run.	Sohan
7	Fixed RTO	<code>--rto fixed:1.5</code> means $RTO = 1.5 \times \text{base RTT}$, where base RTT = emulator delay up + delay down . With <code>--both delay=20</code> that is 40 ms, so <code>fixed:1.5</code> is 60 ms.	Sinjan
8	Result line	At the end of a transfer the sender prints exactly one line beginning <code>RESULT</code> followed by a single JSON object of <code>RunStats</code> fields. Nothing else may print a line starting with <code>RESULT</code> . <code>run_matrix.py</code> reads only that line.	Sohan
9	What goodput means	Goodput = $\text{file bytes} \times 8 \div \text{elapsed seconds}$, where <i>elapsed</i> runs from sending <code>seq 1</code> to receiving the ACK that covers the last data packet — handshake and FIN excluded. Wire throughput = every byte the sender put on the wire, headers and retransmissions included, $\times 8 \div$ the same elapsed time.	Sohan
10	RunStats fields	<code>protocol</code> , <code>window</code> , <code>seqbits</code> , <code>rto_mode</code> , <code>file_bytes</code> , <code>elapsed_ms</code> , <code>goodput_bps</code> , <code>wire_bytes</code> , <code>throughput_bps</code> , <code>data_sent</code> , <code>retransmissions</code> , <code>timeouts</code> , <code>dup_acks</code> , <code>acks_received</code> , <code>corrupt_dropped</code> , <code>sha256_match</code> . The harness adds the run settings itself — <code>seed</code> , <code>loss</code> , <code>dup</code> , <code>corrupt</code> , <code>reorder</code> , <code>delay</code> , <code>jitter</code> .	Sohan
11	Trace lines	<code>us</code> , <code>dir</code> , <code>type</code> , <code>seq</code> , <code>ack</code> , <code>action</code> , <code>delayMs</code> . The <code>type</code> and <code>ack</code> fields are new in the emulator rewrite. Today every ACK is logged with <code>seq 0</code> , so ACKs cannot be told apart — and Experiment 4 needs to tell them apart.	Sohan
12	Logging	No printing per packet unless <code>--verbose</code> is given. Writing a line for every packet costs more than the protocol itself and makes goodput numbers meaningless.	Everyone
13	Tests and main	Every new test class is added to both <code>build.sh</code> and the <code>TESTS</code> line of the <code>Makefile</code> , or it never runs. Run <code>./build.sh</code> before every push: <code>main</code> must always build, because a compile error there stops all four of us.	Everyone

#	TOPIC	PROPOSED DEFAULT	OWNER
14	Sequence arithmetic	One small <code>SeqSpace</code> helper for every modular comparison — distance from the window base, "inside the window", "inside the previous window" — built on <code>Math.floorMod</code> and shared by Go-Back-N and Selective Repeat. The wraparound rule is then written once and tested once.	Sohini

7 · How to test your work

TEST CLASS	WHAT IT MUST PROVE
TimerWheelTest	Timers pop in deadline order; equal deadlines pop in scheduling order; a cancelled timer never fires; cancelling a timer that already fired is harmless; an empty heap reports no deadline; and a randomised sequence of thousands of schedules and cancels matches a simple sorted list — the oracle test, which catches heap bugs that hand-written cases miss.
RttEstimatorTest	The first sample sets $SRTT = R$ and $RTTVAR = R/2$; the four rows of the table in 3.5 to within $1e-9$; an update in the wrong order would fail; each timeout doubles RTO; RTO never goes below the floor or above the cap.
GoBackNTest	Correct transfer over all five profiles; zero retransmissions on a clean channel , which proves the timer stops and restarts correctly; completes with 20% ACK loss; a <code>--seqbits 3</code> , window 7 transfer wraps correctly.
Karn check	After a forced retransmission, the estimator receives no sample for that packet. Easiest to test by counting samples the protocol passes on.

Reuse the fixture. `NetEmSmokeTest` shows how to run the emulator inside a test on free ports. Your Go-Back-N tests can follow the same pattern.

Convention 13: add each new test class to `build.sh` and to the `Makefile`.

8 · When something breaks

SYMPTOM	LIKELY CAUSE	HOW TO CHECK
Retransmissions on a perfect channel	Timer not stopped when <code>base == nextSeq</code> , not restarted on ACK, or RTO far too small	<code>GoBackNTest</code> 's zero-retransmission case
Stalls for ever after one loss	No timer running while packets are outstanding; or a zero timeout passed to <code>setSoTimeout</code>	Log the heap's earliest deadline each loop
CPU at 100% while idle	The loop never blocks: remaining time computed as 0 and not clamped to 1 ms	Print the value given to <code>setSoTimeout</code>
RTO climbs and never comes down	Backoff applied but sampling never resumes after a clean exchange	Log samples accepted and rejected by Karn's rule
Negative RTT samples	Mixing <code>currentTimeMillis</code> with <code>nanoTime</code> , or subtracting in the wrong order	Use one clock everywhere
RTO wrong by a few percent	$SRTT$ updated before $RTTVAR$	The table test in 3.5
<code>ConcurrentModificationException</code>	A firing timer schedules a new one while you iterate the queue	Collect expired timers first, then act
Old data accepted after a wrap	Window too big for <code>--seqbits</code> , or <code>%</code> used where <code>Math.floorMod</code> was needed	The illegal-window demonstration in 3.8
Goodput at large windows far below $W \div RTT$	Printing every packet, or timeouts firing before ACKs return	Run without <code>--verbose</code> ; count timeouts
<code>bash\r: No such file or directory</code> in WSL	<code>build.sh</code> with Windows line endings	Re-clone inside WSL; <code>.gitattributes</code> now forces Unix line endings

9 · Experiment 4 — RTO sensitivity

This is the experiment that makes our submission distinctive. Other groups can say "adaptive RTO seemed better". We can say *how many retransmissions were wasted, and why*, because our emulator's trace records what really happened to every packet.

Question	How many retransmissions are spurious under fixed timeouts versus adaptive RTO, and what does each policy cost in completion time?
Variable	RTO policy: <code>fixed:1.2</code> , <code>fixed:1.5</code> , <code>fixed:2</code> , <code>fixed:3</code> , <code>adaptive</code>
Held fixed	Selective Repeat for labelling, Go-Back-N for completion time; window 32; 2% loss; 20 ms delay each way with 10 ms jitter — jitter is what makes a fixed timeout fire early
Seeds	10 per cell
Measured	Retransmissions, split into <i>necessary</i> , <i>spurious</i> — <i>ACK lost</i> , and <i>spurious</i> — <i>premature timeout</i> ; completion time; goodput

How a retransmission is labelled

1. The sender logs every transmission: sequence number and attempt number.
2. From the emulator trace, take the sender-to-receiver DATA lines for each sequence number, in order. **Skip lines whose action is `dup` or `dup+corrupt`** — those are extra copies the emulator made, not transmissions.
3. For attempt $k \geq 2$ of packet s , look at what happened to attempt $k - 1$:
 - `drop` or `corrupt` — the earlier copy never usefully arrived, so this retransmission was **necessary**.
 - `pass` or `reorder` — the earlier copy arrived intact, so this one was **spurious**. Then check the receiver-to-sender trace: if the ACK for s was dropped or corrupted, label it **spurious — ACK lost**; otherwise **spurious — premature timeout**.

Why Selective Repeat, not Go-Back-N, for labelling. A Go-Back-N receiver *discards* intact packets that arrive out of order, so "the earlier copy arrived" does not mean "the earlier copy was accepted". Resending it may have been genuinely necessary. Selective Repeat buffers every intact packet, so for it the label is exact. Use Go-Back-N only for completion time and total retransmissions, and explain this choice in the report — it is a good viva answer.

This depends on convention 11. The trace only identifies ACKs once Sohan's rewrite adds `type` and `ack` to each line. Today every ACK is logged with `seq 0`. Check it is in place before week 6.

What we expect

- `fixed:1.2` — many premature timeouts, because 10 ms of jitter pushes many round trips past 1.2×40 ms.
- `fixed:3` — almost no spurious retransmissions, but slow recovery from every real loss.
- `adaptive` — few spurious retransmissions *and* completion time near the best fixed setting. If it is not, find out why before writing it up.

Figure 4

- Left panel: stacked bars per policy — necessary, spurious because an ACK was lost, spurious because of a premature timeout.
- Right panel: completion time per policy with 95% confidence intervals.

10 · Your report section — timers, RTO and Go-Back-N

1. **The event loop** — one thread, the heap, and why there is no race.
2. **RTT estimation** — the RFC 6298 update, the worked table, the floor and cap and why they deviate from the RFC.
3. **Karn's algorithm** — the timeline from 3.6.
4. **Go-Back-N** — sender and receiver rules, and the $W \leq 2^k - 1$ bound with its failure case.
5. **Experiment 4** — the labelling method, Figure 4, and what it shows.

About two to three pages.

11 · How we work together

Nightly check-in — by 10 pm, in the group chat

Three lines, every night of the sprint. For example:

```
Done:      receive buffer releases 1,3,4,2 in order; 5 tests passing
Tomorrow:  duplicate handling, then open the PR
Blocked:   nothing
```

In a two-week sprint, a silent day costs whoever is waiting on you half a day. If you are blocked, say so that evening — never save it for the next gate.

Branches

Never commit directly to `main`. Use one branch per task, named after your role — `m2/short-task-name`. Always start from the latest `main`:

```
git checkout main
git pull
git checkout -b m2/short-task-name
```

Bring `main` into your branch at least once a day, so you meet other people's changes while the conflicts are still small:

```
git fetch origin
git merge origin/main
```

Pull requests

Push your branch, open a pull request on GitHub, and use this description:

```
What:      one sentence
Why:      which playbook task this is
How tested: the commands you ran and what they printed
Ask me:    two things you want the reviewer to question
```

WHEN	YOUR REVIEWER — AND WHOSE PRS YOU REVIEW
Sprint A · 14–20 September	Shaili Seth
Sprint B · 21–27 September	Sohan Mandal

Review within 12 hours. An unreviewed PR blocks someone. If your partner is stuck, anyone else may review instead, and you review your partner's work on the same terms.

The explanation check. Before approving, the reviewer asks two or three *why* questions — *why this data structure, what happens if this ACK is lost, why this bound and not another*. If the author cannot answer, the PR does not merge. A five-minute voice call is enough. It rehearses the viva every day, and it catches copy-pasted code while there is still time to rewrite it.

Commits

- At least **one commit every working day** of the sprint, under your own name and email.
- Small commits whose messages say what changed — *"Release buffered packets once the gap at rcv_base fills"*, not *"update"*.
- Never commit on someone else's behalf. The brief counts that as non-contribution by the absent member.
- No Co-Authored-By lines, no bot pull requests, and Dependabot stays off.** The contributor list must show exactly our four names.

Other people's files

If you need a change in a file someone else owns, message them first. If it is urgent and they are unreachable, make the smallest change you can, say so in your PR, and tag them. A merge conflict in a file is resolved by that file's owner.

12 · Using AI without losing marks

PERMITTED BY THE BRIEF

Explaining concepts · debugging · reviewing your code · generating tests · boilerplate and plotting scripts · improving your writing

NOT PERMITTED

Generating the core protocol implementation and submitting it as your own work

Our rule: use AI to understand your module, then type the code yourself. Two reasons, both about marks. The viva is individual and worth 5 of the 10 marks, and sir will ask you to change your code live — code you did not write is code you cannot change under pressure, and it scores zero however well it runs. And a chatbot asked for a textbook protocol gives every group roughly the same code. Another group in our year is doing P3, and the brief treats identical-looking submissions as collusion.

Uses that help

- *"Explain Karn's algorithm with a timeline where the ACK arrives just after a retransmission."* Then close it and draw the timeline yourself.
- *"Here is my window check and a failing test. Why does it fail when the sequence numbers wrap?"* — debugging code you wrote.
- *"Write tests for a receive buffer that must release packets in order."* Tests are explicitly permitted.
- *"Quiz me on my module the way a viva examiner would."* The single most useful thing it can do for you.

Uses that cost marks

- *"Write GoBackN.java."* Even if you read it afterwards, you will not be able to change it live.
- Putting AI output into a PR you cannot then explain in the review check.

Log it the same day

Whenever AI materially helped, add an entry under your role in `AI-USE.md`:

```
### 2026-09-17 – receive buffer
**What:** asked why Java's % operator goes negative for a wrapped sequence number
**Permitted under:** explaining concepts
**Mine:** the buffer design, all of the code, and the tests for the wrap case
```

Honest disclosure carries no penalty. Undisclosed use discovered at the viva does.

13 · Viva preparation

1. How do you avoid a race between a timeout firing and an ACK arriving?

- There is no race: both are events on one thread, handled one at a time.
- Walk through the loop in 3.3; contrast with a timer thread needing locks.

2. Why a heap? And why is it called TimerWheel?

- Few timers, exact ordering, $O(\log n)$ operations; your cancellation choice and its cost.
- A real timing wheel is a bucketed $O(1)$ structure; ours is a heap by choice.

3. Walk me through Jacobson/Karels with numbers.

- First sample, then the update in RFC order; work the second row of the table in 3.5.
- Why RTTVAR must use the old SRTT.

4. Why add four times RTTVAR?

- Timing out at the mean would fire on about half of normal packets; the variance term moves the timeout into the tail.

5. What does Karn's algorithm protect against?

- Draw the 300 ms / 320 ms timeline; the ambiguous ACK; the rule; backoff kept until a clean sample.

6. Why does Go-Back-N resend the whole window? Isn't that wasteful?

- The receiver discards out-of-order packets, so everything after a loss must be resent anyway.
- Yes, it is wasteful under loss — that is what Experiment 1 and 2 show, and why Selective Repeat exists.

7. Why $W \leq 2^k - 1$?

- Sender window plus receiver window must fit in the sequence space. Draw the $k = 2$ case from 3.8.

8. Why `nanoTime` and not `currentTimeMillis`?

- Monotonic versus wall clock; a clock correction could make an RTT negative.

9. Your minimum RTO differs from RFC 6298. Why?

- 1 second is for the Internet; on a 40 ms path it stalls every loss for 25 round trips. State your floor.

10. Live change: shrink the window from 32 to 8 during a transfer.

- New sends respect the new limit immediately; packets already in flight above it are simply acknowledged or resent as normal.

11. Live change: switch from adaptive to fixed RTO.

- A `ProtocolConfig` value; no estimation and no backoff in fixed mode; show where the RTO is read.

14 · Help, references and glossary

Who to ask

QUESTIONS ABOUT	ASK	GITHUB
Packet format, checksum, handshake, FIN, CLIs, Stop-and-Wait, the integrity suite	Shaili Seth	shailiseth24
Timers, RTT estimation, Karn's algorithm, Go-Back-N, fixed and adaptive RTO	Sinjan Mishra	Thesin-01
Selective Repeat, the receive buffer, window bounds, wraparound	Sohini Pandit	sohinix
The emulator, trace files, calibration, RunStats, CSVs, plots, repository access	Sohan Mandal	sohan1611

Stuck for more than an hour? Say so in the group chat. An hour stuck in a two-week sprint is expensive; two minutes of someone else's attention usually is not.

References

- **RFC 1071**, *Computing the Internet Checksum* — section 1 is the algorithm.
- **RFC 6298**, *Computing TCP's Retransmission Timer* — Jacobson/Karels estimation, Karn's algorithm, backoff.
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach* — section 3.4 (reliable data transfer: rdt 3.0, Go-Back-N, Selective Repeat) and section 3.5 (TCP round-trip time estimation).
- **Java 21 API documentation** — [java.nio.ByteBuffer](#), [java.net.DatagramSocket](#), [java.util.PriorityQueue](#), [java.security.MessageDigest](#), [java.util.Random](#), [Math.floorMod](#).
- **In the repository** — [docs/PLAN.md](#) for the design decisions, [docs/member-playbook.html](#) for everyone's dates, [AI-USE.md](#), and the existing tests, which show the house test style.

Glossary

ARQ	Automatic Repeat reQuest: reliability by detecting loss and retransmitting. Stop-and-Wait, Go-Back-N and Selective Repeat are three ARQ schemes.
RTT	Round-trip time — data out, ACK back.
RTO	Retransmission timeout — how long the sender waits for an ACK before resending.
SRTT, RTTVAR	The smoothed RTT and its smoothed variation, from which RTO is computed (RFC 6298).
Cumulative ACK	"Everything up to n has arrived." Used by Go-Back-N.
Selective ACK	"This particular packet has arrived." Used by Selective Repeat.
Window	How many unacknowledged packets the sender may have in flight at once.
Sequence space	The range of sequence numbers: 2^k values for k bits, wrapping back to 0.
Goodput	Useful file bytes delivered per second — excluding headers and retransmissions.
Throughput	Every byte put on the wire per second — including headers and retransmissions.
Spurious retransmission	Resending a packet whose original actually arrived.
Network byte order	Big-endian: most significant byte first. ByteBuffer 's default.
One's complement sum	Addition where a carry out of the top bit is added back in at the bottom — the end-around carry. The basis of the Internet checksum.
Seed	The starting value for the emulator's random generator. Same seed, same decisions, on any machine.
95% confidence interval	A range around a measured mean reflecting run-to-run variation. Every data point we plot carries one.
JIT warm-up	The JVM runs code slowly at first and compiles hot paths as it goes, so the first packets of a run can be slower.

15 · Sprint checklist

- ☐ **Tue 15** — cloned, identity set, build green, first commit merged
- ☐ **Tue 15** — RTT table worked by hand, in both orders
- ☐ **Wed 16** — `docs/timers-design.md` committed
- ☐ **Thu 17** — reviewed Shaili's `Packet` PR
- ☐ **Fri 18** — `TimerWheel` merged with the oracle test; joined the 8 pm call
- ☐ **Sun 20** — `RttEstimator` merged; floor and cap documented
- ☐ **Tue 22** — Go-Back-N transfers at 0% and 10% loss
- ☐ **Wed 23** — Go-Back-N merged: 5 profiles × 10 seeds, wraparound, illegal-window demo
- ☐ **Thu 24** — adaptive RTO in Stop-and-Wait and Selective Repeat, paired with their owners
- ☐ **Fri 25** — `--rto fixed:X` for all three
- ☐ **Sun 27** — `docs/exp4-design.md`
- ☐ **Every night** — three-line check-in by 10 pm; at least one commit every working day